WealthTrack – Investment Portfolio Management System

1. Problem Statement

A wealth management firm needs a digitized system to manage client investment portfolios in order to:

- Eliminate data loss.
- Avoid miscommunication.
- Improve client engagement.

The solution is a **frontend-only Vue.js application** that provides an intuitive way to manage users (clients) and their portfolios.

2. High-Level Logic

A. User Management Logic

- A user represents a **client** in the system.
- Users are stored in a **Vuex store** and persisted in **localStorage**.
- Users have attributes: id, name, email, and optional phone.
- Logic flow:
    - **List Users**: Fetch all users from store → Display in a table.
    - **Add User**: User fills a form → Validation checks (name length, email format, phone optional but regex checked) → Commit to store → Save in localStorage.
    - **Edit User**: On selection, form is pre-filled → Update in store → Persist to localStorage.
    - **Delete User**: Remove by ID from store → Update localStorage.

B. Portfolio Management Logic

- A portfolio represents an **investment account** owned by a client.
- Portfolios are linked to a client using clientId.
- Portfolios have attributes: id, name, status (UPCOMING | ACTIVE | CLOSED), initialInvestment, startDate, notes, holdings[].

- Logic flow:
  - **List Portfolios**:
    - Fetch all portfolios from store.
    - Provide a filter (by status).
    - Display: Portfolio Name, Status, Client Name, Initial Investment.
  - **Add Portfolio**:
    - User fills form fields → Validations (name length, status enum, non-negative number for investment, required date).
    - Portfolio created with unique id and createdAt timestamp.
    - Store commit → Persist in localStorage.
  - **Edit Portfolio**:
    - Load portfolio by ID → Pre-fill form → Update values.
    - Save to store and localStorage.
  - **Delete Portfolio**:
    - Remove by ID from store → Update localStorage.
  - **Portfolio Details**:
    - Retrieve portfolio by ID.
    - Show details: Name, Status, Created Date, Client Name, Initial Investment (formatted), Start Date, Notes.
    - Show holdings table (symbol, quantity, avgPrice).
    - If no holdings → display No holdings.

C. State Management Logic

- **Vuex Store** manages application state.
- **Mutations**: Directly update state (e.g., addUser, updatePortfolio).
- **Actions**: Wrap logic like auto-generating id and timestamps before committing.
- **Getters**: Provide easy access (e.g., getUserById, getPortfolioById, portfoliosByStatus).
- **Persistence**: On every state change, updated data is saved to **localStorage**. On reload, data is loaded back from localStorage.

D. Validation Logic

- **Zod Schemas** define validation rules:
  - User: name length ≥ 2, valid email, optional phone regex.
  - Portfolio: name ≥ 3 chars, status must be one of the enums, initialInvestment must be ≥ 0, startDate required, notes optional.
- **Vee-Validate** integrates Zod with form fields to enforce these rules interactively.

E. UI/UX Logic

- **TailwindCSS** ensures consistent spacing, colors, responsiveness.
- **PrimeVue Icons** add visual clarity (e.g., chart icon for logo, wallet icon for investments).
- **Design decisions**:
  - Portfolios displayed as cards.
  - Investments highlighted in green (visually appealing 💰 format).
  - Empty states (e.g., no holdings or no portfolios) show user-friendly messages.

3. Instructions for Evaluator

Step 1: Clone the Repository

git clone <GITHUB_REPO_LINK>
cd investment-portfolio

Step 2: Install Dependencies

npm install

Step 3: Run Development Server

npm run dev

Visit: http://localhost:5173/

Step 4: Test Features

- Navigate between **Clients** and **Portfolios** using the top menu.
- Try **creating/editing/deleting** both users and portfolios.
- Check **filtering by portfolio status**.
- Open a **portfolio detail page** to view its investments.

4. GitHub Repository

🔗 **GitHub Link:** https://github.com/ektagoel-12/Investment-portfolio-management

5. Future Enhancements

- Add backend APIs to persist data beyond localStorage.
- Add authentication for secure access.
- Add portfolio performance analytics with charts.
- Add automated Jest/Vue Test Utils test cases.